HS PF

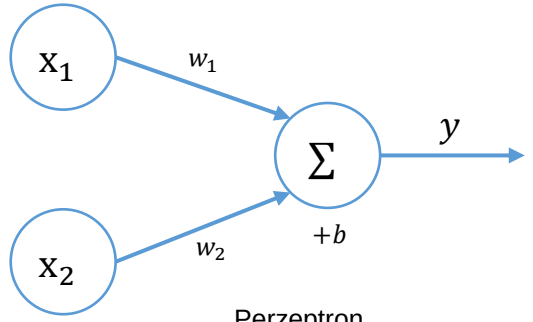
Programmierung in Python

Neuronale Netzwerke in NumPy / Python

Prof. Dr.-Ing. Stefan Kray

Das Perzeptron

- Einfaches Modell für menschliches Neuron
- 1957 von Frank Rosenblatt entwickelt
- Besteht aus zwei oder mehr Eingängen x_i, die gewichtet (w_{ij}) miteinander zu einem Ausgang y verknüpft werden:
  - $y = \begin{cases} 1 & \sum_i (w_i x_i + b) > 0 \\ 0 & sonst \end{cases}$
  - Bias b ist für alle Eingänge gleich



Perzeptron

Beispiel 2 Eingänge:

$$y = (w_1 x_1 + w_2 x_2 + b) > 0$$

Ergebnis ist 0 oder 1

- Beispiel:
  - zwei Eingänge x_1 und x_2
  - Gewichte $w_1 = w_2 = 1$
  - Bias $b = 0$

    → wann ist der Ausgang $o == 1$:

x_1	x_2	o
0	0	0
0	1	1
1	0	1
1	1	1

→ entspricht logischem "Oder" (OR)

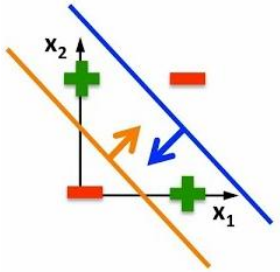
Das Perzeptron

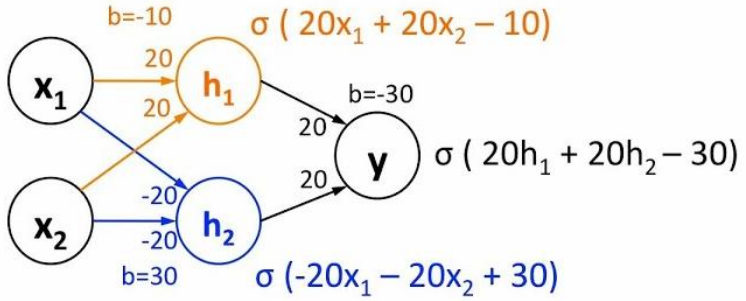
- Für manche Funktionen sind mehrlagige Netze notwendig, z. B. xor (exclusiv oder)
- Die inneren Schichten des neuronalen Netzwerks nennt man "hidden layers"

x_1	x_2	o
0	0	0
0	1	1
1	0	1
1	1	0

Solving XOR with a Neural Net

Linear classifiers cannot solve this





σ(20*0 + 20*0 – 10) ≈ 0
σ(20*1 + 20*1 – 10) ≈ 1
σ(20*0 + 20*1 – 10) ≈ 1
σ(20*1 + 20*0 – 10) ≈ 1

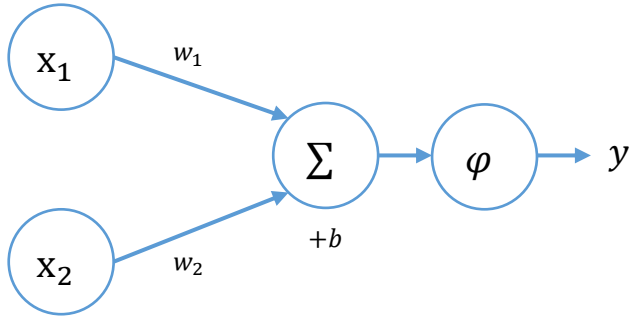
σ (-20*0 – 20*0 + 30) ≈ 1
σ (-20*1 – 20*1 + 30) ≈ 0
σ (-20*0 – 20*1 + 30) ≈ 1
σ (-20*1 – 20*0 + 30) ≈ 1

σ (20*0 + 20*1 – 30) ≈ 0
σ (20*1 + 20*0 – 30) ≈ 0
σ (20*1 + 20*1 – 30) ≈ 1
σ (20*1 + 20*1 – 30) ≈ 1

Quelle:
https://www.youtube.com/watch?v=kNPGXgzxoHw

σ steht hier (näherungsweise) für den "> 0" Vergleich in der Formel links

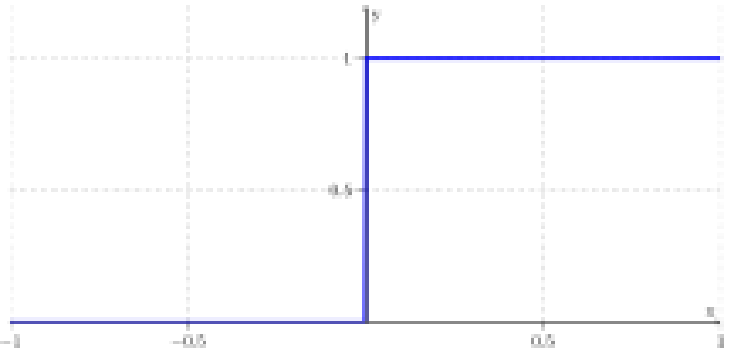
Aktivierungsfunktion φ



- Das Perzeptron / Neuron wird deutlich mächtiger, wenn eine Aktivierungsfunktion hinzugefügt wird
- Die Aktivierungsfunktion φ ist nichtlinear!

$$y = \varphi\left(\sum_i (w_i x_i + b)\right)$$

- Beispiele vorher: Schwellwertfunktion

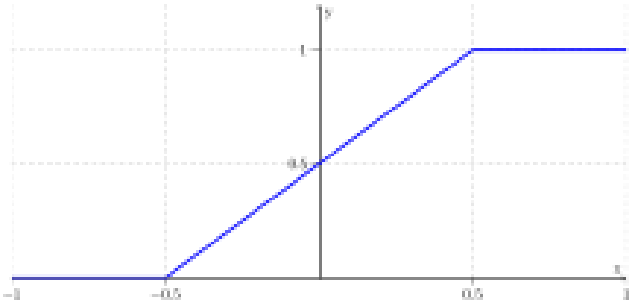


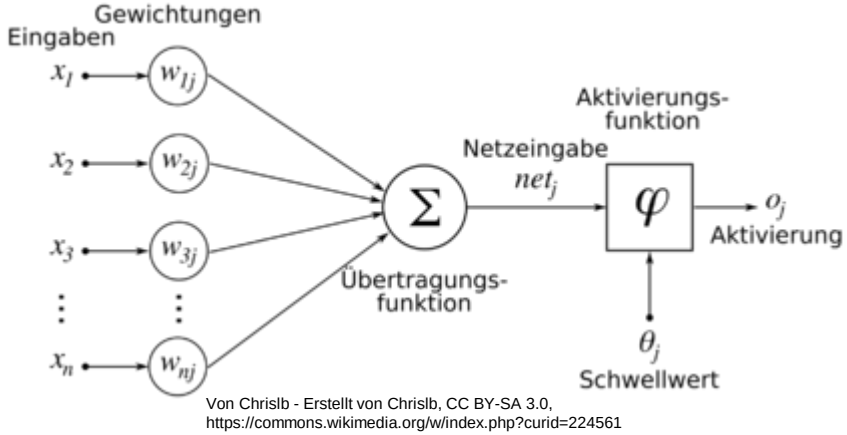
- $\varphi = \begin{cases} 1 & wenn\ v > 0 \\ 0 & sonst \end{cases}$

Aktivierungsfunktion φ

- Stückweise lineare Funktion:

$$\varphi^{\text{pwl}}(v) = \begin{cases} 1 & \text{wenn } v \geq \frac{1}{2} \\ v + \frac{1}{2} & \text{wenn } -\frac{1}{2} < v < \frac{1}{2} \\ 0 & \text{wenn } v \leq -\frac{1}{2} \end{cases}$$





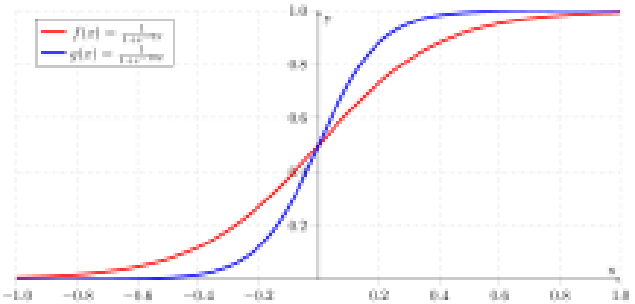
Von Chrislb - Erstellt von Chrislb, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=224561

Darstellung eines künstlichen Neurons nach Wikipedia.
Den Schwellwert θ_j haben wir als Bias b bezeichnet. Man findet auch die Bezeichnung w_0 (0tes Gewicht).

- Sigmoidfunktion:

$$\varphi_a^{\text{sig}}(v) = \frac{1}{1 + \exp(-av)}$$

- bei hohem a geht das in eine Schwellwertfunktion über

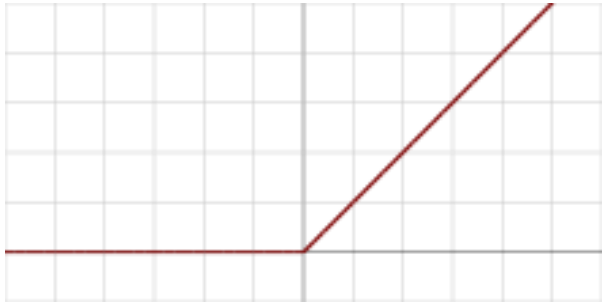


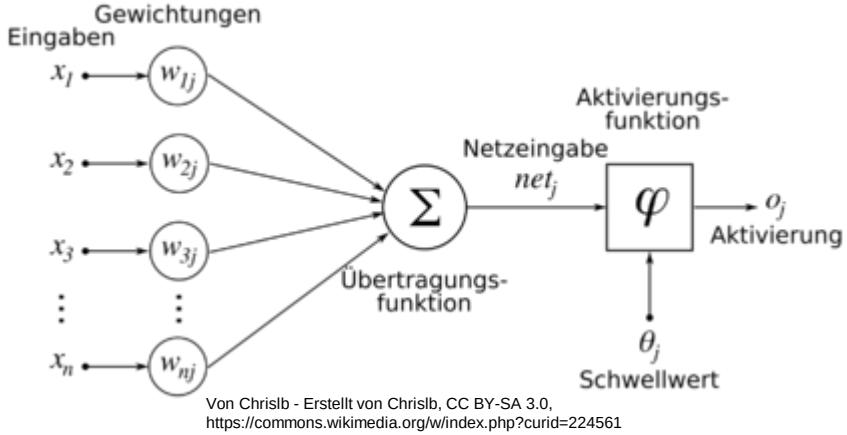
$a = 5$
$a = 10$

Aktivierungsfunktion φ

- Rectifier (ReLu):

$$\varphi(v) = \max(0, v)$$





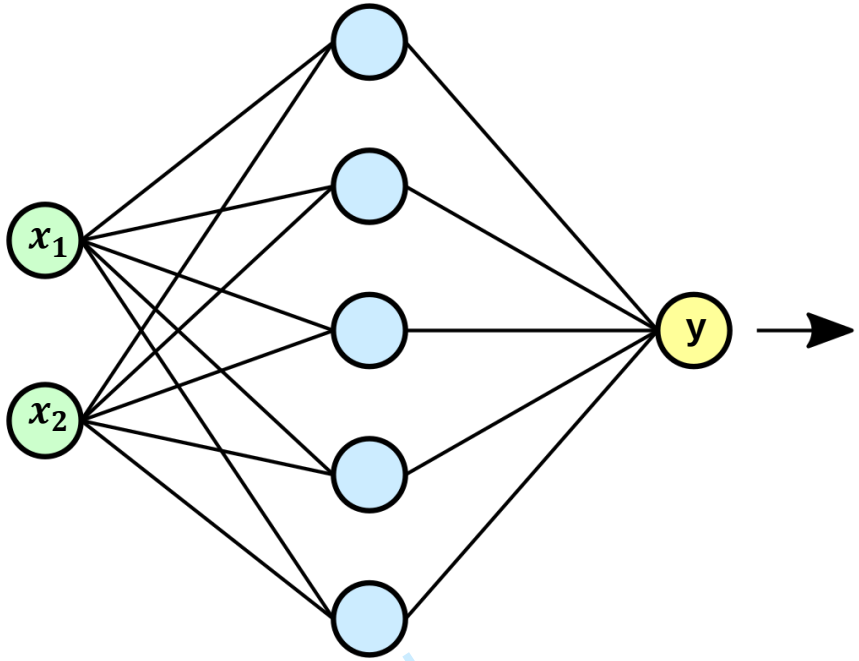
Von Chrislb - Erstellt von Chrislb, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=224561

Darstellung eines künstlichen Neurons nach Wikipedia.
Den Schwellwert θ_j haben wir als Bias b bezeichnet. Man findet auch die Bezeichnung w_0 (0tes Gewicht).

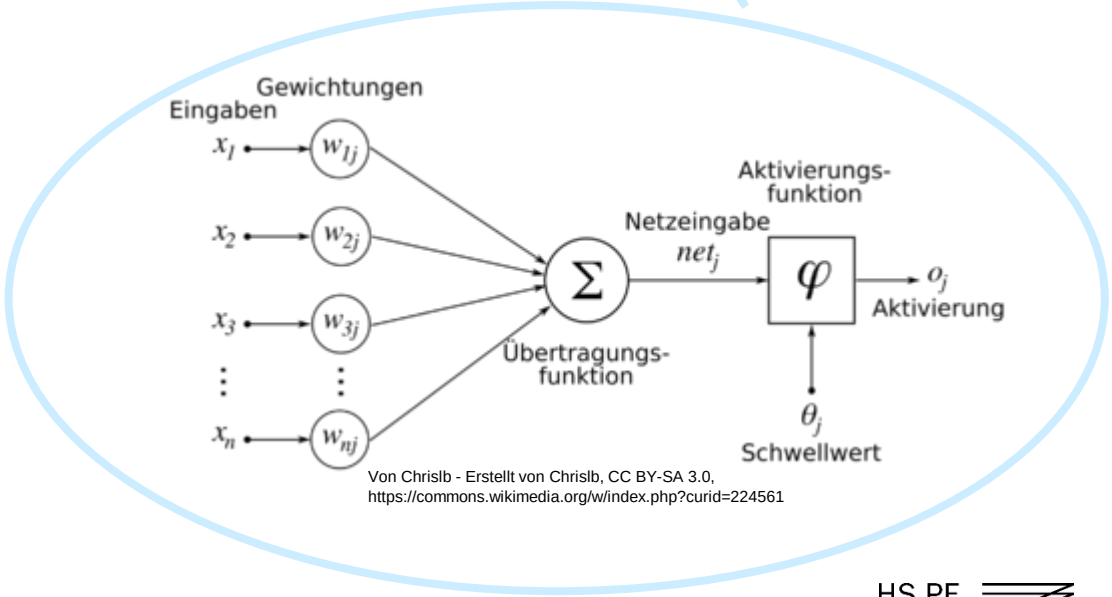
ReLus werden gerne in größeren Netzwerken eingesetzt, in denen σ-Funktionen nicht mehr gut funktionieren.

Neuronales Netz (1)

- Ein Neuron (siehe unten) wird kurz als Kreis dargestellt
- Neuronen werden zu Netzen verschaltet
- Am Ende steht / stehen eine oder mehrere eine Ausgaben
- Neuronen zwischen Eingabeschicht und Ausgabeschicht werden als "Hidden Layer" bezeichnet
- Neuronale Netze gibt es seit den 1950ern!
- In den 70ern wurde das Thema wieder aktuell, konnte aber aufgrund mangelnder Rechenleistung keinen Durchbruch erzielen
- Tiefe neuronale Netze wurden in den 1980ern erforscht
- Seit 2010: Durch Nutzung von GPUs enorme Erfolge erzielt; seit >10 Jahren "Hot Topic"
- Trainieren tiefer neuronaler Netze wird als "**Deep Learning**" bezeichnet!



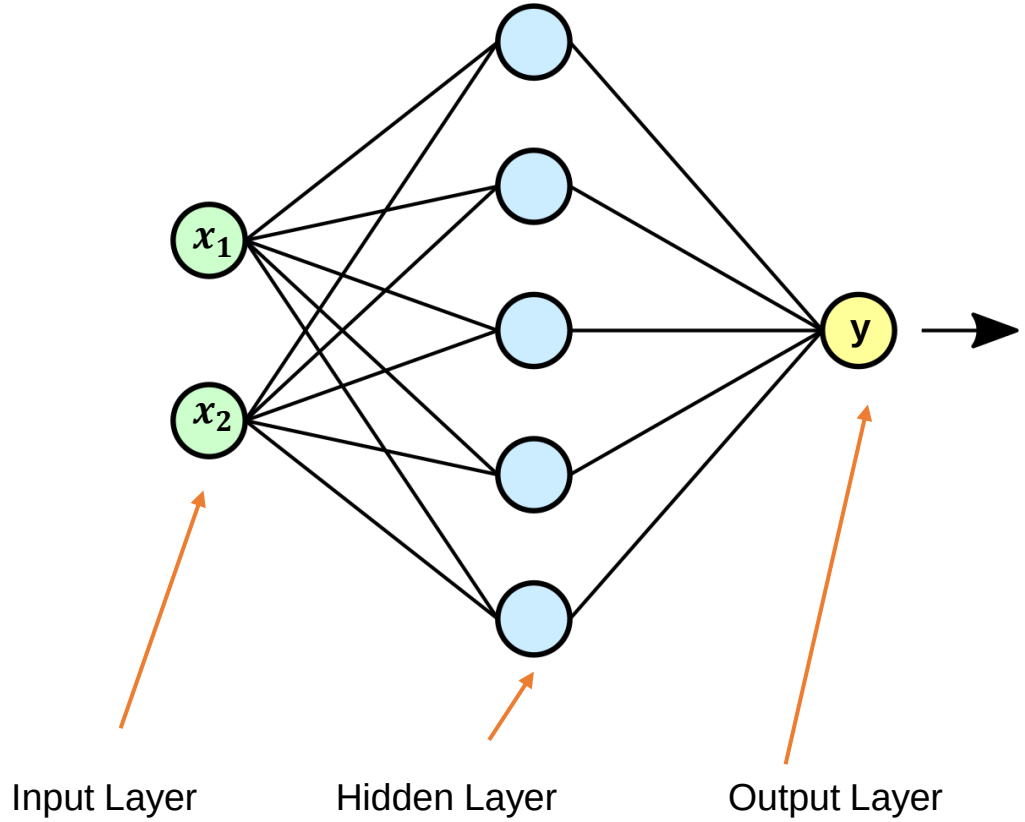
Von Dake, Mysid - Vectorized by Mysid in CorelDraw on an image by Dake., CC BY 1.0, https://commons.wikimedia.org/w/index.php?curid=141?126



Von Chrislb - Erstellt von Chrislb, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=224561

Neuronales Netz (2)

- Konstruktionsregeln (Faustformel):
  - Nicht mehr als 3 Hidden Layers; mit einem Hidden Layer anfangen und sukzessive hinzufügen, falls der Lernerfolg nicht ausreicht
  - Anzahl der Neuronen eines Hidden Layers sollte nicht zu stark vom Vorgänger und Nachfolger abweichen
  - Anzahl der Eingangsknoten >= Anzahl der Parameter des Eingangsvektors
  - Bei Regressionsaufgaben (Fit von Werten) hat der Outputlayer keine Aktivierungsfunktion, d.h. $\varphi(x) = x$ (Identitätsfunktion)



Von Dake, Mysid - Vectorized by Mysid in CorelDraw on an image by Dake., CC BY 1.0, https://commons.wikimedia.org/w/index.php?curid=1412126

Neuronales Netz als Matrix-Vektor Produkt

- Die Gewichte und Offsets der Neuronen werden als Matrix dargestellt
- Ein Layer ist jeweils eine Matrix!
- Die Berechnung eines Neuronalen Netzes ist eine **Folge von Matrix- und Vektormultiplikationen sowie Addition des Bias b**
- Zudem wird die **Aktivierungsfunktion** nach jedem "Layer" angewandt
- Die Schritte werden für alle Layer durchgeführt

$$a_0 = h(w_{00}x_0 + w_{10}x_1 + b_0)$$

$$a_1 = h(w_{01}x_0 + w_{11}x_1 + b_1)$$

$$a_2 = h(w_{02}x_0 + w_{12}x_1 + b_2)$$

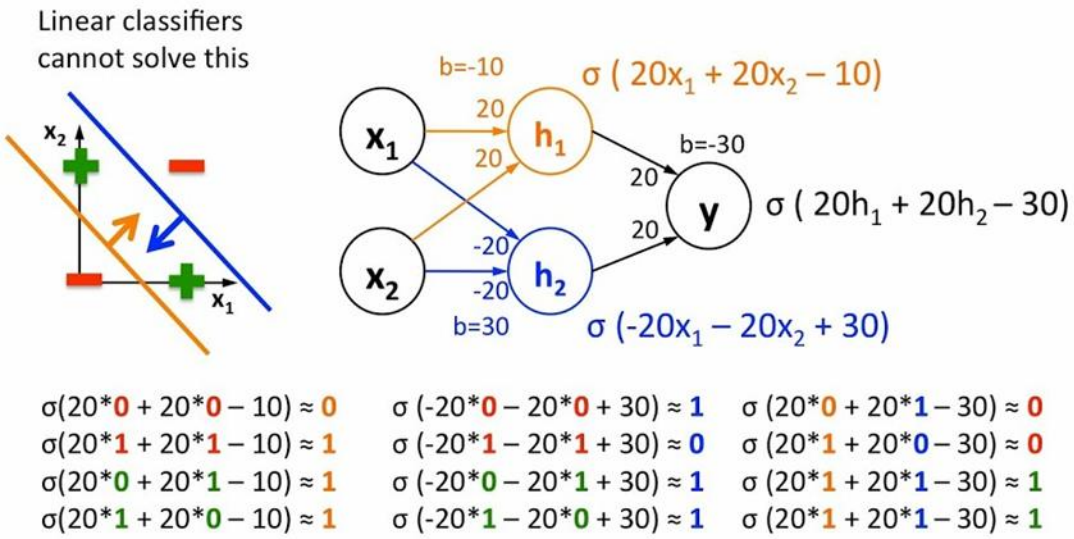
$$\vec{a} = h\left(\begin{bmatrix} w_{00} & w_{10} \\ w_{01} & w_{11} \\ w_{02} & w_{12} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}\right) = h(W\vec{x} + \vec{b})$$

aus Kneusel 2021
In der Darstellung ist h die Aktivierungsfunktion, w_{ij} sind die Gewichte und b die Bias-Werte

Aufgabe xor_nn.py

- Realisieren Sie xor als Neuronales Netz in NumPy
- Laden Sie xor_nn.py
- Vordefiniert sind die Gewichte und Bias-Werte sowie die Sigmoid-Funktion
- Berechnen Sie den Hidden Layer h mit Hilfe von hidden_weights und hidden_b
- Berechnen Sie das Ergebnis mit Hilfe des letzten Schrittes, out_weights und out_b
- Hinweis: Matrix-Matrix oder Matrix-Vektorprodukte rechnen Sie mit $np.dot$

Solving XOR with a Neural Net



https://visualstudiomagazine.com/Articles/2019/07/01/modern-neural-networks.aspx?Page=1

Zusammenfassung

- Neuronale Netze werden durch Matrizen und Vektoren dargestellt
- Einfache Operatoren: Multiplikation, Addition sowie nichtlineare Aktivierungsfunktionen
- "Anwendung" eines Netzes einfach zu implementieren
- Die Frage ist jedoch, wie bestimmt man sinnvolle Gewichte für die einzelnen Layer
  - → Trainieren des Netzes mit Eingangsdaten notwendig